Index

1. interview question
2. mcq question
3. coding question (solve any 10 from 20)

Interview Question

Q1. What is collection Framework & Why use it?
Q2. What are the benefits of collection over the array?
Q3. Explain collection hierarchy?
Q4. Explain Iterable interface in detail?
Q5. Explain the collection interface with its methods?
Q6. Explain types of Collection like List,Set,Queue with its differences ?
Q7. Explain List interface with its methods?
Q8. Explain Vector class and Why Vector is thread safe and legacy ?
Q9. Explain cursors in the collection and why use it and explain its name?
Q10. Explain Iterator interface?
Q11. Explain Enumeration interface?
Q12. Explain ListIterator interface?
Q13. Explain the difference between Iterator and Enumeration?
Q14. Explain the difference between Iterator and ListIterator?
Q15. Explain for each for fetch collection?
Q16. How to store user defined objects in a collection and why do we need to store it?
Q17. What is FailSafe and FailFast in collection?
Q18. Explain the ArrayList class?
Q19. Explain the difference between Vector and ArrayList?
Q20. Explain the LinkedList class of List interface?

MCQ Question

Q1. What will be the output?

```
Collection<Integer> c = new ArrayList<>();
c.add(10);
c.add(20);
System.out.println(c.size());
```

A) 0
B) 1

C) 2
D) Compilation error

Answer: C
Explanation: Collection is the root interface; ArrayList implements it. Two elements → size = 2.

---

2. Collections vs Array

Q2. Which line will cause error?

```
int[] arr = new int[2];
arr[0] = 10;
arr[1] = 20;
arr[2] = 30;
```

A) arr[0]
B) arr[1]
C) arr[2]
D) None

Answer: C
Explanation: Arrays are fixed-size → index out of bounds. Collections grow dynamically.

---

3. Collection Hierarchy

Q3. Which is NOT part of Collection hierarchy?
A) List
B) Set
C) Map
D) Queue

Answer: C
Explanation: Map is part of the framework but not a subtype of Collection.

---

4. Iterable Interface

Q4. What does this print?

```
List<Integer> list = List.of(1,2,3);
for(Integer i : list){
   System.out.print(i);
}
```

A) Error
B) 123
C) 321
D) Null

Answer: B
Explanation: Iterable allows enhanced for-loop.

---

5. Collection Interface Methods

Q5. Output?

```
List<Integer> list = new ArrayList<>();
list.add(10);
list.add(20);
list.remove(Integer.valueOf(10));
System.out.println(list);
```

A) [10,20]
B) [20]
C) [10]
D) Error

Answer: B
Explanation: Removes object 10 → list becomes [20].

---

6. List vs Set vs Queue

Q6. Output?

```
Set<Integer> set = new HashSet<>();
set.add(10);
set.add(10);
System.out.println(set.size());
```

A) 2
B) 1
C) 0
D) Error

Answer: B
Explanation: Set does not allow duplicates.

---

7. List Interface

Q7. Output?

```
List<String> list = new ArrayList<>();
list.add("A");
list.add(0,"B");
System.out.println(list);
```

A) [A,B]
B) [B,A]
C) [A]
D) Error

Answer: B
Explanation: Index-based insertion.

---

8. Vector Class

Q8. Which is TRUE about Vector?
A) Not synchronized
B) Thread-safe
C) Faster than ArrayList
D) Deprecated

Answer: B
Explanation: Vector methods are synchronized → thread-safe but slower.

---

9. Cursors

Q9. Which is NOT a cursor?
A) Iterator
B) Enumeration
C) ListIterator
D) Scanner

Answer: D
Explanation: Scanner is for input, not traversal.

---

10. Iterator Interface

Q10. Output?

```
List<Integer> list = new ArrayList<>(List.of(1,2,3));
Iterator<Integer> it = list.iterator();
while(it.hasNext()){
   System.out.print(it.next());
}
```

A) 123
B) 321
C) Error
D) Null

Answer: A

---

11. Enumeration Interface

Q11. Which method belongs to Enumeration?
A) hasNext()
B) next()
C) hasMoreElements()
D) remove()

Answer: C

---

12. ListIterator

Q12. Output?

```
List<Integer> list = List.of(1,2,3);
ListIterator<Integer> it = list.listIterator();
System.out.println(it.hasPrevious());
```

A) true
B) false
C) Error
D) Null

Answer: B
Explanation: Cursor at start → no previous element.

---

13. Iterator vs Enumeration

Q13. Which feature is in Iterator but not Enumeration?
A) Traversal
B) remove()
C) Forward only
D) Works on Vector

Answer: B

---

14. Iterator vs ListIterator

Q14. Which is TRUE?
A) Iterator supports backward traversal
B) ListIterator supports backward traversal
C) Both same
D) None

Answer: B

---

15. For-each Loop

Q15. Output?

```
List<String> list = List.of("A","B");
for(String s : list){
   s = s + "X";
}
System.out.println(list);
```

A) [AX, BX]
B) [A, B]
C) Error
D) Null

Answer: B
Explanation: For-each does not modify original collection.

---

16. User-defined Objects

Q16. What is required to store objects in HashSet properly?
A) toString()
B) equals() & hashCode()
C) constructor
D) finalize()

Answer: B

---

17. Fail-Fast vs Fail-Safe

Q17. Output?

```
List<Integer> list = new ArrayList<>(List.of(1,2,3));
for(Integer i : list){
   list.add(4);
}
```

A) Runs fine
B) Infinite loop

C) ConcurrentModificationException
D) Null

Answer: C
Explanation: Fail-fast behavior.

---

18. ArrayList

Q18. Output?

```
List<Integer> list = new ArrayList<>();
list.add(1);
list.add(2);
System.out.println(list.get(1));
```

A) 1
B) 2
C) Error
D) Null

Answer: B

---

19. Vector vs ArrayList

Q19. Which is faster?
A) Vector
B) ArrayList
C) Both same
D) None

Answer: B
Explanation: ArrayList is non-synchronized → faster.

---

20. LinkedList

Q20. Output?

```
LinkedList<Integer> list = new LinkedList<>();
list.addFirst(10);
list.addLast(20);
System.out.println(list);
```

A) [20,10]
B) [10,20]
C) Error
D) Null

Answer: B

Coding Questions

---

1. Find Maximum Element using ArrayList

Problem: Store integers in an ArrayList and find the maximum element.
Requirement: Do not use array.
Example:
Input: [3, 7, 2, 9, 5]
Output: 9

---

2. Reverse Elements in ArrayList

Problem: Reverse the elements stored in an ArrayList.
Example:
Input: [1, 2, 3, 4]
Output: [4, 3, 2, 1]

---

3. Sum of Elements in ArrayList

Problem: Calculate the sum of all elements in an ArrayList.
Example:
Input: [1, 2, 3, 4]
Output: 10

---

4. Find Even Numbers in ArrayList

Problem: Extract all even numbers from an ArrayList into another list.
Example:
Input: [1, 2, 3, 4, 5]
Output: [2, 4]

---

5. Count Occurrence of Element

Problem: Count how many times a specific number appears in an ArrayList.
Example:
Input: [1, 2, 2, 3, 2], target = 2
Output: 3

---

6. Find Second Largest Element

Problem: Find the second largest element in an ArrayList.
Example:
Input: [10, 20, 4, 45, 99]
Output: 45

---

7. Remove Duplicate Elements (ArrayList Only)

Problem: Remove duplicate values from an ArrayList without using Set.
Example:
Input: [1, 2, 2, 3, 4, 4]
Output: [1, 2, 3, 4]

---

8. Merge Two ArrayLists

Problem: Combine two ArrayLists into a single ArrayList.
Example:
Input: [1, 2] and [3, 4]
Output: [1, 2, 3, 4]

---

9. Find Missing Number

Problem: Given numbers from 1 to N in an ArrayList with one missing, find the missing number.
Example:
Input: [1, 2, 4, 5]
Output: 3

---

10. Check Palindrome using ArrayList

Problem: Check if elements in an ArrayList form a palindrome.
Example:
Input: [1, 2, 3, 2, 1]
Output: Palindrome

◆ Part B: 3 Coding Problems (POJO Class with Collection)

---

11. Store and Display Student Objects

Problem: Create a Student object (id, name, marks) and store multiple objects in an ArrayList. Display all details.
Example:
Input: 3 students
Output:
Student1 → id=1, name=A, marks=80
Student2 → id=2, name=B, marks=75

---

12. Find Topper using ArrayList

Problem: From a list of Student objects, find the student with highest marks.
Example:
Input: marks = [70, 85, 90]
Output: Student with 90 marks

---

13. Remove Duplicate Objects using Set

Problem: Store Employee objects (id, name) and remove duplicates based on id using Set or LinkedHashSet.
Example:
Input: IDs [101, 102, 101]
Output: Unique IDs [101, 102]

◆ Part C: 7 Coding Problems (ArrayList, Set, LinkedHashSet)

---

14. Remove Duplicates using Set

Problem: Convert an ArrayList into a Set to remove duplicates.
Example:
Input: [1, 2, 2, 3, 4, 4]
Output: [1, 2, 3, 4]

---

15. Maintain Order using LinkedHashSet

Problem: Remove duplicates but maintain insertion order using LinkedHashSet.
Example:
Input: [3, 1, 2, 3]
Output: [3, 1, 2]

---

16. Sort Elements in ArrayList

Problem: Sort elements in ascending order.
Example:
Input: [5, 2, 8, 1]
Output: [1, 2, 5, 8]

---

17. Find Common Elements

Problem: Find common elements between two ArrayLists and store result in a Set.
Example:

Input: [1, 2, 3] and [2, 3, 4]
Output: [2, 3]

---

18. Frequency Count using Collection

Problem: Count frequency of each element using collections.
Example:
Input: [1, 2, 2, 3]
Output: {1=1, 2=2, 3=1}

---

19. Convert ArrayList to LinkedHashSet

Problem: Convert an ArrayList into LinkedHashSet and observe ordering.
Example:
Input: [5, 3, 5, 2]
Output: [5, 3, 2]

---

20. Filter Elements

Problem: Remove all elements greater than a given value using collection methods.
Example:
Input: [1, 5, 8, 3], value = 4
Output: [1, 3]